



Repaso: Tecnologías Avanzadas de Desarrollo

PHP Orientado a Objetos

Licencia



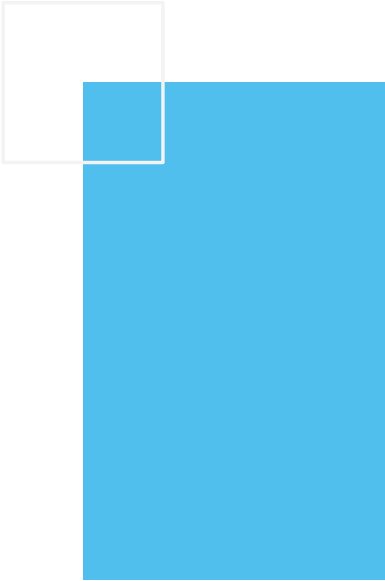
Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

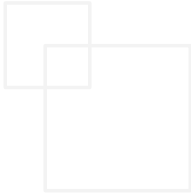
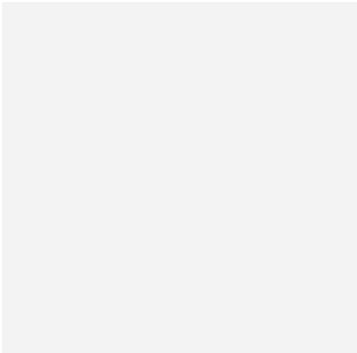
<https://creativecommons.org/licenses/by-nc-nd/4.0/>

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.



ÍNDICE



- 
1. PHP
 2. Clases y Objetos
 3. Constructores
 4. Clases estáticas
 5. Herencia
 6. Clases Abstractas y Finales
 7. Interfaces
 8. Métodos
 9. Espacios de Nombres
 10. Autoload
 11. Gestión de errores
 12. Excepciones
 13. SPL
- 

Clases y objetos

PHP sigue un paradigma de programación orientada a objetos (POO) basada en clases.

Una clase es una plantilla que define las propiedades y métodos para poder crear objetos. De esta manera, un objeto es una instancia de una clase.

Tanto las propiedades como los métodos se definen con una visibilidad (quién puede acceder):

- Privado - **private**: Sólo puede acceder la propia clase.
- Protegido - **protected**: Sólo puede acceder la propia clase o sus descendientes.
- Público - **public**: Puede acceder cualquier otra clase.

Para declarar una clase, se utiliza la palabra clave **class** seguido del nombre de la clase. Para instanciar un objeto a partir de la clase, se utiliza **new**:



Clases con mayúscula
Todas las clases empiezan por letra mayúscula.

ej1

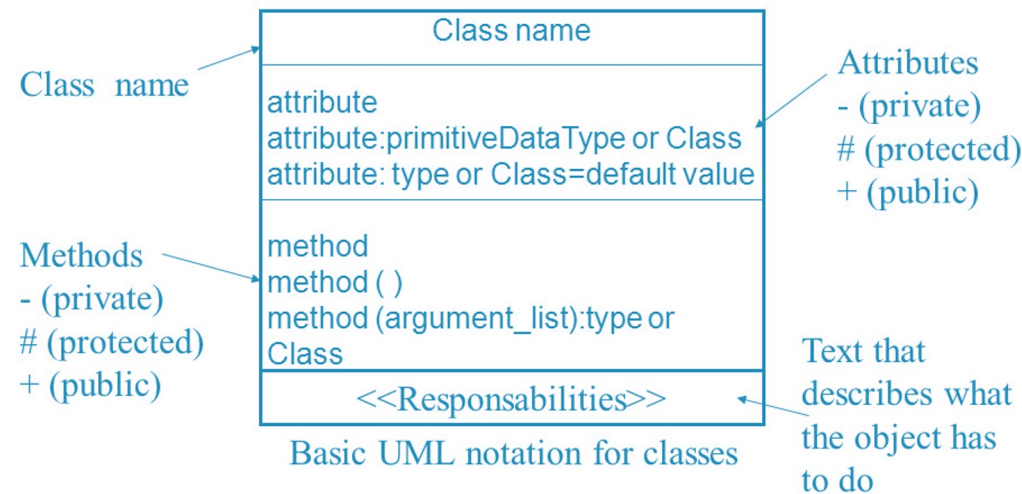
```
<?php
class NombreClase {
    // propiedad
    // y
    // métodos
    $ob = new NombreClase();
```

Recordamos: UML

Cuando un proyecto crece, es normal modelar las clases mediante UML (¿recordáis Entornos de Desarrollo?). Las clases se representan mediante un cuadrado, separando el nombre, de las propiedades y los métodos:

Static Structure Diagrams

- **Class Diagram:** shows classes & relationships
(Klassen-Diagramm & Beziehungen)



Clases y objetos

Una vez que hemos creado un objeto, se utiliza el operador `->` para acceder a una propiedad o un método:

```
$objeto->propiedad;  
$objeto->método(parámetros);
```

Si desde dentro de la clase, queremos acceder a una propiedad o método de la misma clase, utilizaremos la referencia `$this`:

```
$this->propiedad;  
$this->método(parámetros);
```

Como ejemplo, codificaríamos una persona en el fichero `Persona.php`:

Aunque se pueden declarar varias clases en el mismo archivo, es una mala práctica. Así pues, cada fichero contendrá una sola clase, y se nombrará con el nombre de la clase.

ej2: Persona.php

```
<?php  
class Persona{  
  
    private String $nombre;  
  
    public function setNombre(String $nom){  
        $this -> nombre=$nom;  
    }  
  
    public function imprimir(){  
        echo $this->nombre;  
        echo " ";  
    }  
}  
  
$hector = new Persona();  
$hector->setNombre("Héctor");  
$hector->imprimir();
```

Encapsulación

Las propiedades se definen privadas o protegidas (si queremos que las clases heredadas puedan acceder).

Para cada propiedad, se añaden métodos públicos (getter/setter):

```
public setPropiedad(tipo $param)
public getPropiedad() : tipo
```

Las constantes se definen públicas para que sean accesibles por todos los recursos.

ej3:

```
<?php
class MayorMenor{
    private int $mayor;
    private int $menor;
    public function getMayor(){
        return $this->mayor;
    }
    public function setMayor($mayor){
        $this->mayor = $mayor;
        return $this;
    }
    public function getMenor(){
        return $this->menor;
    }
    public function setMenor($menor){
        $this->menor = $menor;
        return $this;
    }
}
```

Recibiendo y enviando objetos

Es recomendable indicarlo en el tipo de parámetros. Si el objeto puede devolver nulos se pone `?` delante del nombre de la clase.



Objetos por referencia

Los objetos que se envían y reciben como parámetros siempre se pasan por referencia.

ej4.php

```
<?php
include_once("../ej3/MayorMenor.php");
function mayormenor(array $nums) : ?MayorMenor{
    $a = max($nums);
    $b = min($nums);

    $res = new MayorMenor();
    $res->setMayor($a);
    $res->setMenor($b);
    return $res;
}

$resultado= mayormenor([2,58,6,389,35,39,27,42,22]);
echo "<br> Mayor: ". $resultado->getMayor();
echo "<br> Menor: ". $resultado->getMenor();
```


Constructor

El constructor de los objetos se define mediante el método mágico `__construct`. Puede o no tener parámetros, pero sólo puede haber un constructor.

ej5: Persona.php

```
<?php
class Persona{

    private String $nombre;

    public function __construct(String $nom)
    {
        $this->nombre= $nom;
    }

    public function imprimir(){
        echo $this->nombre;
        echo " ";
    }
    <br>

}

$hector = new Persona("Héctor");
$hector->imprimir();
```

Constructores en php 8

Una de las grandes novedades que ofrece PHP 8 es la [simplificación de los constructores](#) con parámetros, lo que se conoce como promoción de las propiedades del constructor.

Para ello, en vez de tener que declarar las propiedades como privadas o protegidas, y luego dentro del constructor tener que asignar los parámetros a estas propiedades, el propio constructor promociona las propiedades.

Veámoslo mejor con un ejemplo. Imaginemos una clase Punto donde queramos almacenar sus coordenadas:

En PHP 8, quedaría del siguiente modo (mucho más corto, lo que facilita su legibilidad):

ej6:

```
class Punto {  
  
    protected float $x;  
    protected float $y;  
    protected float $z;  
  
    public function __construct(  
        float $x = 0.0,  
        float $y = 0.0,  
        float $z = 0.0  
    ) {  
        $this->x = $x;  
        $this->y = $y;  
        $this->z = $z;  
    }  
}
```

ej6:

```
class Punto2{  
    public function __construct(  
        protected float $x,  
        protected float $y,  
        protected float $z)  
    {}  
}
```

El Orden Importa

A la hora de codificar el orden de los elementos debe ser:

```
<?php
declare(strict_types=1);

class NombreClase {
    // propiedades

    // constructor

    // getters - setters

    // resto de métodos
}
?>
```

`strict_types=1`: desde PHP 7 lo usamos para controlar que la inicialización de las variables coincida siempre con el tipo de dato que le insertamos, si no fuese así obtendríamos un `TypeError`.

Clases estáticas

Son aquellas que tienen propiedades y/o métodos estáticos (también se conocen como de clase, por que su valor se comparte entre todas las instancias de la misma clase).

- Se declaran con static y se referencian con ::.
- Si queremos acceder a un método estático, se antepone el nombre de la clase: Producto::nuevoProducto().
- Si desde un método queremos acceder a una propiedad estática de la misma clase, se utiliza la referencia `self::` self::\$numProductos

Se le llama **Paamayim Nekudotayim**, operador de resolución de ámbito o doble dos puntos "::" (*double colon*) al operador que permite acceder a constantes y a métodos estáticos, además de poder sobrescribir propiedades o métodos de una clase.

ej7:

```
<?php
class Producto {
    const IVA = 0.23;
    private static $numProductos = 0;

    public static function nuevoProducto() {
        self::$numProductos++;
    }
}

Producto::nuevoProducto();
$impuesto = Producto::IVA;
```

Clases estáticas

También podemos tener clases normales que tengan alguna propiedad estática:

ej7: Producto2.php

```
<?php
class Producto2{
    const IVA = 0.21;
    private static $numProductos =0;
    private $codigo;
    public function __construct(String $cod){

        self::$numProductos++;
        $this->codigo=$cod;
    }
    public function mostrarResumen() : String {
        return "El producto ". $this->codigo." es el número " . self::$numProductos;
    }
} // End class

$prod1 = new Producto2("PlayStation 5");
$prod2 = new Producto2("Nintengo Switch");
$prod3 = new Producto2("XBOX Series X");
$prod4 = new Producto2("Steam Deck");
echo $prod2->mostrarResumen();
```

Introspección

Al trabajar con clases y objetos, existen un conjunto de funciones ya definidas por el lenguaje que permiten obtener información sobre los objetos:

- `instanceof`: permite comprobar si un objeto es de una determinada clase
- `get_class`: devuelve el nombre de la clase
- `get_declared_class`: devuelve un array con los nombres de las clases definidas
- `class_alias`: crea un alias
- `class_exists` / `method_exists` / `property_exists`: true si la clase / método / propiedad está definida
- `get_class_methods` / `get_class_vars` / `get_object_vars`: Devuelve un array con los nombres de los métodos / propiedades de una clase / propiedades de un objeto que son accesibles desde dónde se hace la llamada. (Ver Ejemplo 8)

Clonado

Al asignar dos objetos no se copian, se crea una nueva referencia. Si queremos una copia, hay que clonarlo mediante el método `clone(object) : object`

Si queremos modificar el clonado por defecto, hay que definir el método mágico `__clone()` que se llamará después de copiar todas las propiedades.

Más información en <https://www.php.net/manual/es/language.oop5.cloning.php>

Herencia

- PHP soporta herencia simple, de manera que una clase solo puede heredar de otra, no de dos clases a la vez. Para ello se utiliza la palabra clave extends. Si queremos que la clase A hereda de la clase B haremos: `class A extends B`
- El hijo hereda los atributos y métodos públicos y protegidos.

Cada clase en un archivo

Como ya hemos comentado, deberíamos colocar cada clase en un archivo diferente para posteriormente utilizarlo mediante include. En los siguientes ejemplos los hemos colocado juntos para facilitar su legibilidad.

- Por ejemplo, tenemos una clase Item y una Tv que hereda de Item (ver ejemplo 9)

ej9: herencia.php

```
<?php
class Item {
    public $codigo;
    public $nombre;
    public $nombreCorto;
    public $PVP;

    public function mostrarResumen() {
        echo "<p>Prod:". $this->codigo. "</p>";
    }
}

class Tv extends Item {
    public $pulgadas;
    public $tecnologia;
}
```

Herencia

Podemos utilizar las siguientes funciones para averiguar si hay relación entre dos clases:

- `get_parent_class(object)`: string
- `is_subclass_of(object, string)`: bool

ej9: herencia.php

```
$t = new Tv();
$t->codigo = 33;
if ($t instanceof Item) {
    echo $t->mostrarResumen();
}

$padre = get_parent_class($t);
echo "<br>La clase padre es: " . $padre;
$objetoPadre = new $padre;
echo $objetoPadre->mostrarResumen();

if (is_subclass_of($t, 'Item')) {
    echo "<br>Soy un hijo de Item";
}
```


Herencia

SOBRESCRIBIR MÉTODOS

- Podemos crear métodos en los hijos con el mismo nombre que el padre, cambiando su comportamiento. Para invocar a los métodos del padre -> `parent::nombreMetodo()`

ej9: herencia.php

```
class Tv extends Item {  
    public $pulgadas;  
    public $tecnologia;  
  
    /* SOBRESCRIBIR UN MÉTODO DEL PADRE:*/  
  
    public function mostrarResumen() {  
        parent::mostrarResumen();  
        echo "<p>TV ".$this->tecnologia." de ".$this->pulgadas."</p>";  
    }  
}
```

Herencia

CONSTRUCTOR EN HIJOS

- En los hijos no se crea ningún constructor de manera automática. Por lo que si no lo hay, se invoca automáticamente al del padre. En cambio, si lo definimos en el hijo, hemos de invocar al del padre de manera explícita.

```
antiguo.php

<?php
class Producto {
    public string $codigo;

    public function __construct(string $codigo) {
        $this->codigo = $codigo;
    }

    public function mostrarResumen() {
        echo "<p>Prod:". $this->codigo."</p>";
    }
}

class Tv extends Producto {
    public $pulgadas;
    public $tecnologia;

    public function __construct(string $codigo, int $pulgadas, string $tecnologia) {
        parent::__construct($codigo);
        $this->pulgadas = $pulgadas;
        $this->tecnologia = $tecnologia;
    }

    public function mostrarResumen() {
        parent::mostrarResumen();
        echo "<p>TV ". $this->tecnologia." de ". $this->pulgadas."</p>";
    }
}
```

```
ej10: nuevo.php

<?php
class Prod{
    public function __construct(private string $codigo) { }
    public function mostrarResumen() {
        echo "<p>Prod:". $this->codigo."</p>";
    }
}

class Tele extends Prod{
    public function __construct(
        String $codigo,
        private int $pulgadas,
        private String $tecnologia){
        parent::__construct($codigo);
    }

    public function mostrarResumen() {
        parent::mostrarResumen();
        echo "<p>TV ". $this->tecnologia." de ". $this->pulgadas."</p>";
    }
}
```

Clases abstractas

- Las clases abstractas obligan a heredar de una clase, ya que no se permite su instanciación. Se define mediante `abstract class NombreClase {`.
- Una clase abstracta puede contener propiedades y métodos no-abstractos, y/o métodos abstractos.
- Cuando una clase hereda de una clase abstracta, obligatoriamente debe implementar los métodos que tiene el padre marcados como abstractos.

ej11: CocheAbstract.php

```
<?php
abstract class CocheAbstract{
    public function getRuedas(){
        return 4;
    }
    abstract public function setPotencia($potencia);
    abstract public function getPotencia();
}
```

ej11: Audi.php

```
class Audi extends CocheAbstract{
    public $brand = 'Audi';
    protected $potencia;
    public function setPotencia($potencia){
        $this->potencia = $potencia;
    }
    public function getPotencia(){
        return $this->potencia;
    }
}
```

Clases finales

- Son clases opuestas a abstractas, ya que evitan que se pueda heredar una clase o método para sobrescribirlo.

ej12: Electrodomestico.php

```
<?php
class Electrodomestico{
    private $codigo;
    public function getCodigo(){
        return $this->codigo;
    }
    final public function mostrarResumen(): String {
        return "Producto ".$this->codigo;
    }
}
```

ej12: Microondas.php

```
<?php
include_once("Electrodomestico.php");
final class Microondas extends Electrodomestico{
    private $potencia;
    public function getPotencia() : int {
        return $this->potencia;
    }
    // No podemos implementar mostrarResumen()
}
// No podemos heredar de Microondas
```

Interfaces

- Permite definir un contrato con las firmas de los métodos a cumplir. Así pues, sólo contiene declaraciones de funciones y todas deben ser públicas.
- Se declaran con la palabra clave `interface` y luego las clases que cumplan el contrato lo realizan mediante la palabra clave `implements`.
- Se permite la herencia de interfaces. Además, una clase puede implementar varias interfaces (en este caso, sí soporta la herencia múltiple, pero sólo de interfaces).

ej13: interfaces.php

```
<?php
interface Mostrable{
    public function mostrarResumen(): string;
}
interface MostrableTodo extends Mostrable{
    public function mostrarTodo(): string;
}
interface Facturable{
    public function generarFactura(): string;
}

class Producto3 implements MostrableTodo, Facturable{
    // Implementaciones de los métodos

    // Obligatoriamente deberá implementar public function mostrarResumen, most
    rarTodo y generarFactura

    public function mostrarResumen(): string{
        return "resumen";
    }
    public function mostrarTodo(): string{
        return "mostrar";
    }
    public function generarFactura(): string{
        return "factura";
    }
}
```

Métodos encadenados

- Sigue el planteamiento de la programación funcional, y también se conoce como *method chaining*.
- Plantea que sobre un objeto se realizan varias llamadas.
- Para facilitarlo, vamos a modificar todos sus métodos mutadores (que modifican datos, setters, ...) para que devuelvan una referencia a \$this:

ej14: chaining.php

```
<?php
include_once("Libro.php");
$p1 = new Libro();
$p1->setNombre("Harry Potter");
$p1->setAutor("JK Rowling");
echo $p1;
echo "<br>";
// Method chaining
$p2 = new Libro();
$p2->setNombre("La Bestia")->setAutor("Carmen Mola");
echo $p2;
```

ej14: Libro.php

```
<?php
class Libro {
    private $nombre;
    private $autor;
    public function getNombre() : string{
        return $this->nombre;
    }
    public function setNombre($nombre){
        $this->nombre = $nombre;
        return $this;
    }
    public function getAutor() : string{
        return $this->autor;
    }
    public function setAutor($autor){
        $this->autor = $autor;
        return $this;
    }
    public function __toString(): string{
        return $this->nombre." de ".$this->autor;
    }
}
```

Métodos mágicos

Todas las clases PHP ofrecen un conjunto de métodos, también conocidos como *magic methods* que se pueden sobrescribir para sustituir su comportamiento. Algunos de ellos ya los hemos utilizado. Ante cualquier duda, es conveniente consultar la documentación oficial.

Los más destacables son:

- `__construct()` → constructor
- `__destruct()` → se invoca al perder la referencia. Se utiliza para cerrar una conexión a la BD, cerrar un fichero, ...
- `__toString()` → representación del objeto como cadena. Es decir, cuando hacemos `echo $objeto` se ejecuta automáticamente este método.
- `__get(propiedad)`, `__set(propiedad, valor)` → Permitiría acceder a las propiedad privadas, aunque siempre es más legible/mantenible codificar los getter/setter.
- `__isset(propiedad)`, `__unset(propiedad)` → Permite averiguar o quitar el valor a una propiedad.
- `__sleep()`, `__wakeup()` → Se ejecutan al recuperar (`unserialize()`) o almacenar un objeto que se serializa (`serialize()`), y se utilizan para permite definir qué propiedades se serializan.
- `__call()`, `__callStatic()` → Se ejecutan al llamar a un método que no es público. Permiten sobrecargar métodos.

Espacios de nombres

Desde PHP 5.3 y también conocidos como Namespaces, permiten organizar las clases/interfaces, funciones y/o constantes de forma similar a los paquetes en Java.

- Se declaran en la primera línea mediante la palabra clave namespace seguida del nombre del espacio de nombres asignado (cada subnivel se separa con la barra invertida \):
- Por ejemplo, para colocar la clase Producto dentro del namespace Dwes\Ejemplos lo haríamos así:

```
<?php
namespace Dwes\Ejemplos;

const IVA = 0.21;

class Producto {
    public $nombre;

    public function muestra() : void {
        echo"<p>Prod:" . $this->nombre . "</p>";
    }
}
```

Recomendación:

Un sólo namespace por archivo y crear una estructura de carpetas respetando los niveles/subniveles (igual que se hace en Java)

Espacios de nombres

Acceso:

- Para referenciar a un recurso que contiene un namespace, primero hemos de tenerlo disponible haciendo uso de include o require. Si el recurso está en el mismo namespace, se realiza un acceso directo (se conoce como acceso sin cualificar).

Realmente hay tres tipos de acceso:

- sin cualificar: recurso
- cualificado: rutaRelativa\recurso → no hace falta poner el namespace completo.
- totalmente cualificado: \rutaAbsoluta\recurso

```
<?php
namespace Dwes\Ejemplos;

include_once("Producto.php");

echo IVA; // sin cualificar
echo Utilidades\IVA;
// acceso cualificado. Daría error, no existe \Dwes\Ejemplos\Utilidades\IVA
echo \Dwes\Ejemplos\IVA; // totalmente cualificado

$p1 = new Producto();
// lo busca en el mismo namespace y encuentra \Dwes\Ejemplos\Producto
$p2 = new Model\Producto();
// daría error, no existe el namespace Model. Está buscando \Dwes\Ejemplos\Model\Producto

$p3 = new \Dwes\Ejemplos\Producto(); // \Dwes\Ejemplos\Producto
```

Espacios de nombres

Para evitar la referencia cualificada podemos declarar el uso mediante use (similar a hacer import en Java). Se hace en la cabecera, tras el namespace:

Los tipos posibles son:

- use const nombreCualificadoConstante
- use function nombreCualificadoFuncion
- use nombreCualificadoClase
- use nombreCualificadoClase as NuevoNombre // para renombrar elementos

Por ejemplo, si queremos utilizar la clase \Dwes\Ejemplos\Producto desde un recurso que se encuentra en la raíz, haríamos:

```
<?php
include_once("Dwes\Ejemplo\Producto.php");

use const Dwes\Ejemplos\IVA;
use \Dwes\Ejemplos\Producto;

echo IVA;
$p1 = new Producto();
```

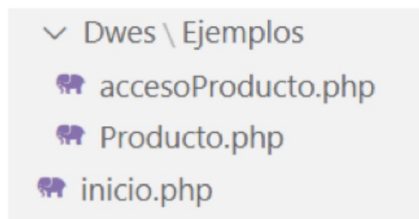
To use or not to use

En resumen, use permite acceder sin cualificar a recursos que están en otro *namespace*. Si estamos en el mismo espacio de nombre, no necesitamos use.

Espacios de nombres

Organización:

- Todo proyecto, conforme crece, necesita organizar su código fuente. Se plantea una organización en la que los archivos que interactúan con el navegador se colocan en el raíz, y las clases que definamos van dentro de un namespace (y dentro de su propia carpeta src o app).



Dwes > Ejemplos > 🐘 Producto.php > ...

```
1  <?php
2  namespace Dwes\Ejemplos;
3
4  const IVA = 0.21;
5
6  class Producto {
7      public $nombre;
8
9      public function muestra() : void {
10         print "<p>Prod:" . $this->nombre . "</p>";
11     }
12 }
```

🐘 inicio.php > ...

```
1  <?php
2  include_once("Dwes/Ejemplos/Producto.php");
3
4  use const Dwes\Ejemplos\IVA;
5  use Dwes\Ejemplos\Producto;
6
7  echo IVA;
8  $p1 = new Producto();
```

Espacios de nombres

Organización, includes y usos:

- Colocaremos cada recurso en un fichero aparte.
- En la primera línea indicaremos su namespace (si no está en el raíz).
- Si utilizamos otros recursos, haremos un `include_once` de esos recursos (clases, interfaces, etc...).
- Cada recurso debe incluir todos los otros recursos que referencie: la clase de la que hereda, interfaces que implementa, clases utilizadas/recibidas como parámetros, etc...
- Si los recursos están en un espacio de nombres diferente al que estamos, emplearemos `use` con la ruta completa para luego utilizar referencias sin cualificar.

Autoload

- ¿No es tedioso tener que hacer el include de las clases? El autoload viene al rescate.
- Así pues, permite cargar las clases (no las constantes ni las funciones) que se van a utilizar y evitar tener que hacer el `include_once` de cada una de ellas. Para ello, se utiliza la función [spl_autoload_register](#)

```
<?php
spl_autoload_register( function( $nombreClase ) {
    include_once $nombreClase.'.php';
} );
?>
```

Por qué se llaman *autoload*?

Porque antes se realizaba mediante el método mágico `__autoload()`, el cual está *deprecated* desde PHP 7.2

Autoload

Y ¿cómo organizamos ahora nuestro código aprovechando el autoload?

- Para facilitar la búsqueda de los recursos a incluir, es recomendable colocar todas las clases dentro de una misma carpeta. Nosotros la vamos a colocar dentro de `app` (más adelante, cuando estudiemos Laravel veremos el motivo de esta decisión). Otras carpetas que podemos crear son `test` para colocar las pruebas PHPUnit que luego realizaremos, o la carpeta `vendor` donde se almacenarán las librerías del proyecto (esta carpeta es un estándar dentro de PHP, ya que Composer la crea automáticamente, ya lo veremos más adelante).
- Como hemos colocado todos nuestros recursos dentro de `app`, ahora nuestro `autoload.php` (el cual colocamos en la carpeta raíz) sólo va a buscar dentro de esa carpeta:

```
<?php
spl_autoload_register( function( $nombreClase ) {
    include_once "app/".$nombreClase.'.php';
} );
?>
```

Autoload y rutas erróneas

En *Ubuntu* al hacer el *include* de la clase que recibe como parámetro, las barras de los namespace (`\`) son diferentes a las de las rutas (`/`). Por ello, es mejor que utilicemos el fichero `autoload`:

```
<?php
spl_autoload_register( function( $nombreClase ) {
    $ruta = "app\\".$nombreClase.'.php';
    $ruta = str_replace("\\", "/", $ruta); // Sustituimos las barras
    include_once $ruta';
} );
?>
```

Gestión de errores

- PHP clasifica los errores que ocurren en diferentes niveles. Cada nivel se identifica con una constante. Por ejemplo:
- `E_ERROR`: errores fatales, no recuperables. Se interrumpe el script.
- `E_WARNING`: advertencias en tiempo de ejecución. El script no se interrumpe.
- `E_NOTICE`: avisos en tiempo de ejecución.
- Podéis comprobar el listado completo de constantes de <https://www.php.net/manual/es/errorfunc.constants.php>

Gestión de errores

Para la configuración de los errores podemos hacerlo de dos formas:

A nivel de php.ini:

- `error_reporting`: indica los niveles de errores a notificar
- `error_reporting = E_ALL & ~E_NOTICE` -> Todos los errores menos los avisos en tiempo de ejecución.
- `display_errors`: indica si mostrar o no los errores por pantalla. En entornos de producción es común ponerlo a off

Mediante código con las siguientes funciones:

- `error_reporting(codigo)` -> Controla qué errores notificar
- `set_error_handler(nombreManejador)` -> Indica que función se invocará cada vez que se encuentre un error. El manejador recibe como parámetros el nivel del error y el mensaje
- A continuación tenemos un ejemplo mediante código:

ej16: errores.php

```
<?php
error_reporting(E_ALL & ~E_NOTICE & ~E_WARNING);
$resultado = $dividendo / $divisor;

error_reporting(E_ALL & ~E_NOTICE);
set_error_handler("miManejadorErrores");
$resultado = $dividendo / $divisor;
restore_error_handler(); // vuelve al anterior

function miManejadorErrores($nivel, $mensaje) {
    switch($nivel) {
        case E_WARNING:
            echo "<strong>Warning</strong>: $mensaje.<br/>";
            break;
        default:
            echo "Error de tipo no especificado: $mensaje.<br/>";
    }
}
```


Excepciones

- La gestión de excepciones forma parte desde PHP 5. Su funcionamiento es similar a *Java*, haciendo uso de un **bloque try / catch / finally**. Si detectamos una situación anómala y queremos lanzar una excepción, deberemos realizar `throw new Exception` (adjuntando el mensaje que lo ha provocado).

ej17: excepciones.php

```
<?php
try {
    if ($divisor == 0) {
        throw new Exception("División por cero.");
    }
    $resultado = $dividendo / $divisor;
} catch (Exception $e) {
    echo "Se ha producido el siguiente error: ".$e->getMessage();
}
```

- La clase `Exception` es la clase padre de todas las excepciones. Su constructor recibe `mensaje[,codigoError][,excepcionPrevia]`.
- A partir de un objeto `Exception`, podemos acceder a los métodos `getMessage()` y `getCode()` para obtener el mensaje y el código de error de la excepción capturada.
- El propio lenguaje ofrece un conjunto de excepciones ya definidas, las cuales podemos capturar (y lanzar desde PHP 7). Se recomienda su consulta en la documentación oficial.

Creando excepciones

- Para crear una excepción, la forma más corta es crear una clase que únicamente herede de Exception.
- Si queremos, y es recomendable dependiendo de los requisitos, podemos sobrecargar los métodos mágicos, por ejemplo, sobrecargando el constructor y llamando al constructor del padre, o rescribir el método `__toString` para cambiar su mensaje:

ej17: MiExcepcion.php

```
<?php
class MiExcepcion extends Exception {
    public function __construct($msj, $codigo = 0, Exception $previa = null
) {
    // código propio
    parent::__construct($msj, $codigo, $previa);
    }
    public function __toString() {
        return __CLASS__ . ": [{$this->code}]: {$this->message}\n";
    }
    public function miFuncion() {
        echo "Una función personalizada para este tipo de excepción\n";
    }
}
```

Excepciones

- Si definimos una excepción de aplicación dentro de un namespace, cuando referenciamos a Exception, deberemos referenciarla mediante su nombre totalmente cualificado (\Exception), o utilizando use:

ej17: referenciando.php

```
<?php
namespace \Dwes\Ejemplos;
class AppExcepcion extends \Exception {}
?>
```

ej17: referenciando.php

```
<?php
namespace \Dwes\Ejemplos;

use Exception;

class AppExcepcion extends Exception {}

?>
```

Excepciones múltiples

- Se pueden usar excepciones múltiples para comprobar diferentes condiciones. A la hora de capturarlas, se hace de más específica a más general.
- Si en el mismo catch queremos capturar varias excepciones, hemos de utilizar el operador `|`:

ej17: excepcionesMultiples.php

```
<?php
class MainException extends Exception {}
class SubException extends MainException {}

try {
    throw new SubException("Lanzada SubException");
} catch (MainException | SubException $e) {
    echo "Capturada Exception " . $e->getMessage();
}
```

ej17: excepcionesMultiples.php

```
<?php
$email = "ejemplo@ejemplo.com";
try {
    // Comprueba si el email es válido
    if(filter_var($email, FILTER_VALIDATE_EMAIL) === FALSE) {
        throw new MiExcepcion($email);
    }
    // Comprueba la palabra ejemplo en la dirección email
    if(strpos($email, "ejemplo") !== FALSE) {
        throw new Exception("$email es un email de ejemplo no válido");
    }
} catch (MiExcepcion $e) {
    echo $e->miFuncion();
} catch (Exception $e) {
    echo $e->getMessage();
}
```

Excepciones

- Desde PHP 7, existe el tipo Throwable, el cual es un interfaz que implementan tanto los errores como las excepciones, y nos permite capturar los dos tipos a la vez:

```
ej17: throwable.php

<?php
try {
    // tu codigo
} catch (Throwable $e) {
    echo 'Forma de capturar errores y excepciones a la vez';
}
```

- Si sólo queremos capturar los errores fatales, podemos hacer uso de la clase Error:

```
ej17: throwable.php

<?php
try {
    // Genera una notificación que no se captura
    echo $variableNoAsignada;
    // Error fatal que se captura
    funcionQueNoExiste();
} catch (Error $e) {
    echo "Error capturado: " . $e->getMessage();
}
```

Relanzar excepciones

- En las aplicaciones reales, es muy común capturar una excepción de sistema y lanzar una de aplicación que hemos definido nosotros. También podemos lanzar las excepciones sin necesidad de estar dentro de un try/catch.

ej17: relanzar.php

```
<?php
class AppException extends Exception {}

try {
    // Código de negocio que falla
} catch (Exception $e) {
    throw new AppException("AppException: ".$e->getMessage(), $e->getCode(), $e);
}
```

SPL

Standard PHP Library es el conjunto de funciones y utilidades que ofrece PHP, como:

- Estructuras de datos
- Pila, cola, cola de prioridad, lista doblemente enlazada, etc...
- Conjunto de iteradores diseñados para recorrer estructuras agregadas, arrays, resultados de bases de datos, árboles XML, listados de directorios, etc.
- Podéis consultar la documentación en <https://www.php.net/manual/es/book.spl.php> o ver algunos ejemplos en <https://diego.com.es/tutorial-de-la-libreria-spl-de-php>
- También podéis consultar la documentación de estas excepciones en <https://www.php.net/manual/es/spl.exceptions.php>
- También define un conjunto de excepciones que podemos utilizar para que las lancen nuestras aplicaciones:
 - LogicException (extends Exception)
 - BadFunctionCallException
 - BadMethodCallException
 - DomainException
 - InvalidArgumentException
 - LengthException
 - OutOfRangeException
 - RuntimeException (extends Exception)
 - OutOfBoundsException
 - OverflowException
 - RangeException
 - UnderflowException
 - UnexpectedValueException

Bibliografía

Páginas web que han ayudado en la construcción de estas diapositivas:

- <https://github.com/olga3emes/PHP-Web/wiki>
- <http://phpdevenezuela.github.io/php-the-right-way/>
- <https://aitor-medrano.github.io/>
- <https://www.mclibre.org/consultar/php/>
- <https://www.php.net/manual/es/index.php>
- Manual de OO en PHP - www.desarrolloweb.com
- Ediciones ENI: PHP (Acceso restringido)

¿Preguntas?

Olga M. Moreno Martín

